SW Engineering CSC648/848 Fall 2023

Section 02

Team 02

Milestone 2

California SafetyNet

Lukas – lheber@sfsu.edu – Team Lead, Github Master

Marshall – back end lead

Danial – front end lead

Ely, Mark, Sanjith

Date: 10/21/2023

Date submitted: 10/3/2023

Date revised: 10/21/2023

Content

1. Functional requirements - prioritized 3

2. UI Mockups and Storyboards 5

3. High level Architecture, Database Organization 9

4. High Level UML Diagrams 15

5. Risks 22

6. Project management 23

1. Functional requirements - prioritized

1. Adherence to Regulations: Software shall have a resource link page to county regulations for data security and privacy.
2. Administration Control Panel: Administrators shall be able to access an interface where they may control users, content, and notifications.
3. Alert Initiation: Administrators shall be able to set off notifications depending on predetermined parameters, such as COVID-19 infection rates, wildfire evacuation levels, and security issues.
4. Community Forum: Shall give people a place to contribute information and take part in community conversations.
5. Crowdsourced Data Submission: Users shall be able to input crowdsourced data, such as information about roadblocks, detours, and security issues.
6. Customizable Alerts: Users shall be able to change alert choices, including the alert categories and transmission methods.
7. Data Integration in Real Time: Shall be able to obtain current information on COVID-19, weather patterns, and wildfires by integrating with governmental organizations.
8. Embedded messaging: Users shall be able to communicate during emergencies by sending and receiving messages within the platform via in-site messaging.
9. Emergency Resource Directory: Shall offer a thorough listing of all available emergency services, including hospitals, fire stations, police stations, and shelters for evacuees.
10. Entry for County Metrics: COVID-19, weather, wildfire, and security metrics shall be entered and updated by county departments (Health, Security, Weather, and Fire).
11. Feedback Mechanism: Shall implement a feedback system so that users may report problems and make suggestions for changes.
12. Help Center: Shall offer a thorough help center with FAQs and instruction manuals.
13. Mapping Interface: Shall interact with interactive maps to view emergency occurrences, metrics, and county-specific data.
14. Marketing Integration: Shall develop marketing plans and collaborate with regional government organizations to increase the uptake of the platform.
15. Multilingual Support: Shall support several languages to serve California's multicultural population.
16. Multi-Platform Support: Shall improve usability and accessibility by expanding the service to mobile apps and making sure consumers can access it on a variety of hardware and software.
17. Privacy Settings: Users shall be able to control their privacy preferences and decide what information they want to make available with others.
18. User Login: Users shall be able to register.
19. User Access: Users shall be able to access the system by logging in.
20. User Registration: Users shall be able to register for accounts only by providing a working email address as confirmation in order to access the platform's features.

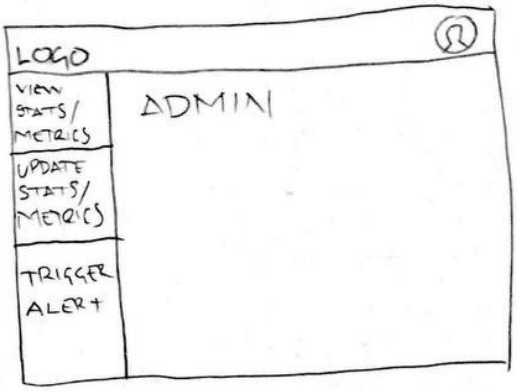
In the following table each requirement is referenced by its unique number and assigned a priority from P1 (must be implemented) to P3 (opportunistic):

REQUIREMENT	PRIORITY
1	P1
3	P1
6	P1
10	P1
11	P1
12	P1
13	P1
2	P2
7	P2
15	P2
16	P2
17	P2
4	P3
5	P3
8	P3
9	P3
14	P3
18	P3
19	P3
20	P3

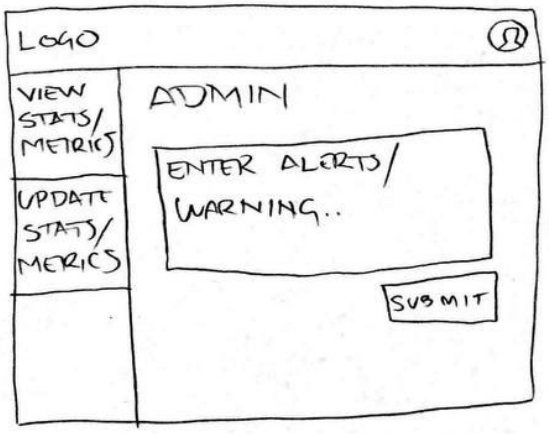
2. UI Mockups and Storyboards

ADMIN USE CASE:

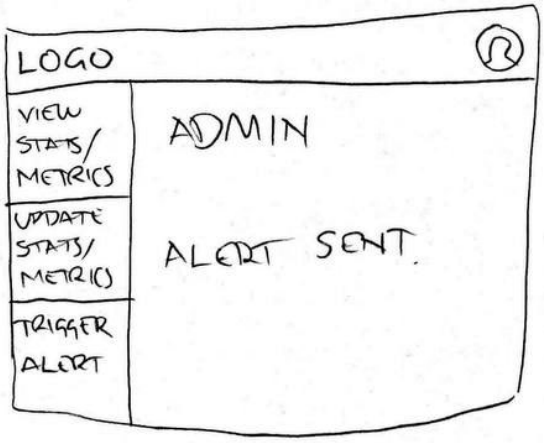
TRIGGER ALERT



ADMIN CLICKS ON TRIGGER ALERT.

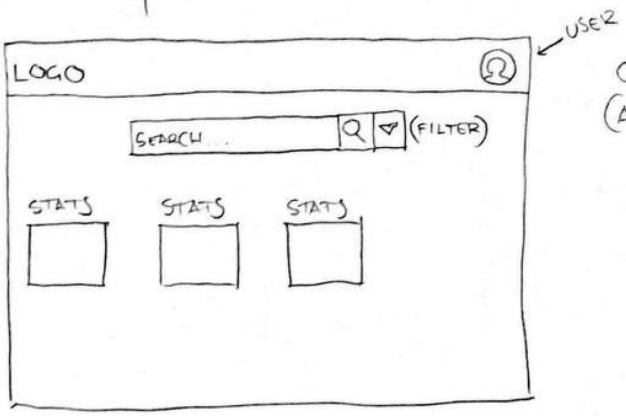


ADMIN ENTERS ALERT MESSAGE AND SUBMITS.

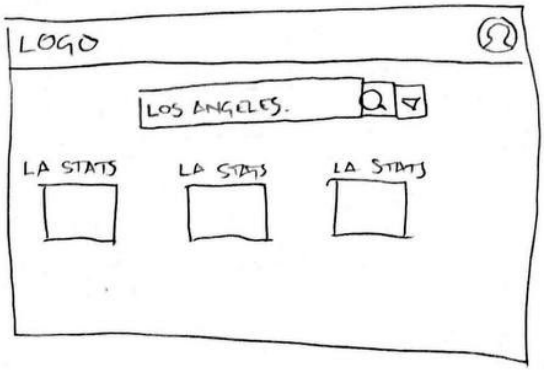


ALERT SENT

GENERAL PUBLIC USE CASE:
PLANNING A TRIP

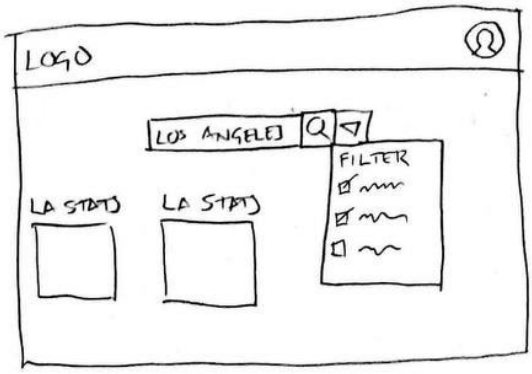


OPENS APP
(ASSUME LOGGED IN)



SEARCH LOS ANGELES

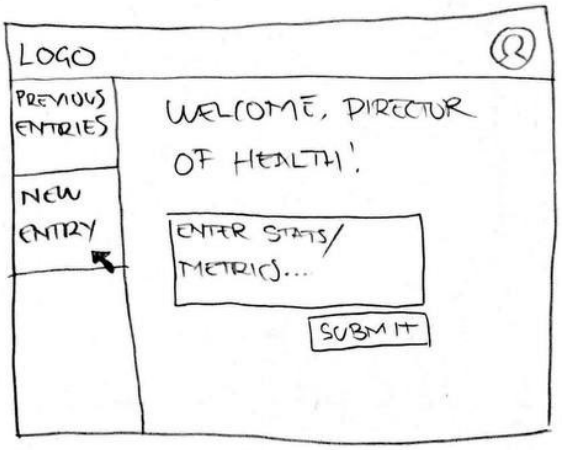
DISPLAYS ALL
"LOS ANGELES" METRICS
AND STATUS.



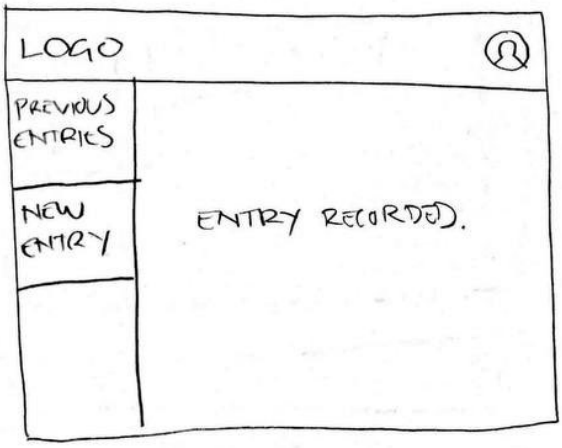
USER USES FILTER.
SEARCH RESULTS ARE
REFINED.

USER DESIDES IF HE/SHE
WANTS TO GO TO
LOS ANGELES.

COUNTY DIRECTOR USE CASE:
ENTERING STATUS/METRICS

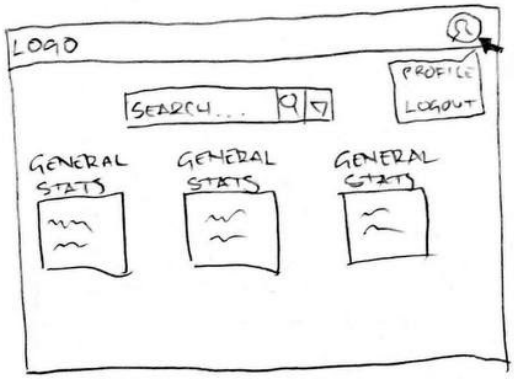


COUNTY DIRECTOR ENTERS STATS/METRICS AND SUBMITS.

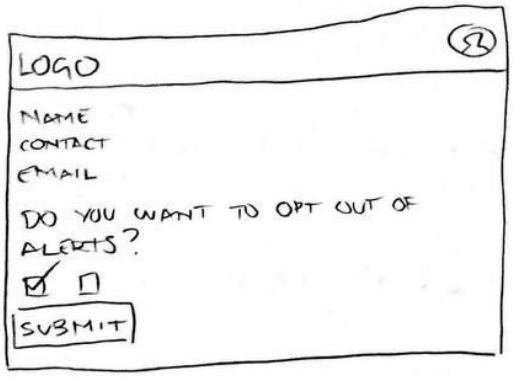


GENERAL PUBLIC USE CASE:

OPT OUT OF ALERTS



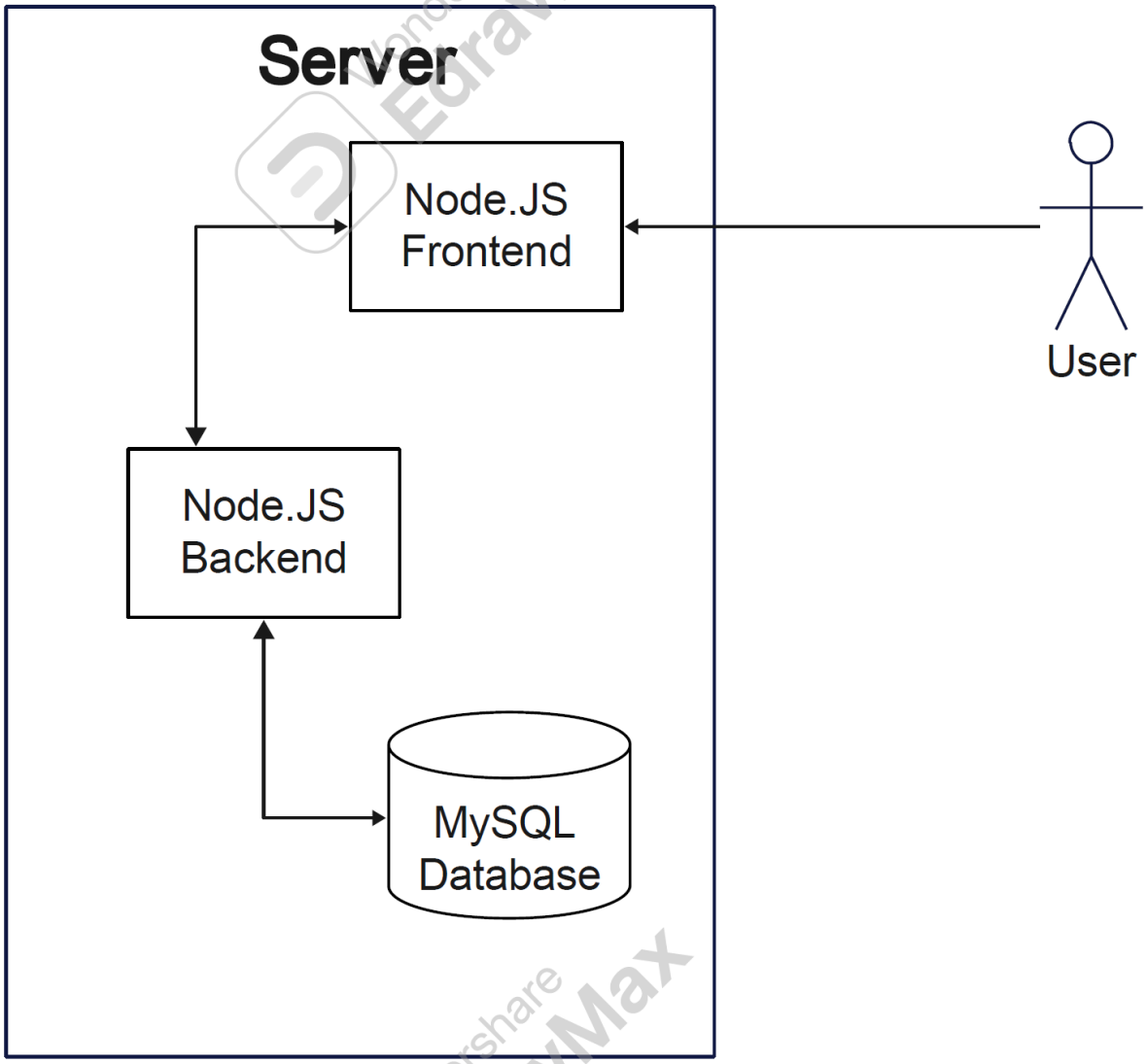
USER CLICKS ON "PROFILE"



USER CHECKS "YES" BOX AND SUBMITS.

USER IS NOW OPT OUT OF ALERTS.

3. High level Architecture, Database Organization



1. User: User Table has all the user details which are collected when user email is registered/verified

   A user belongs to one county (identified by county ID or name). County has multiple users

   - User ID (Primary Key) – INT
   - Password - VARCHAR (255)
   - Email Address - VARCHAR(55)
   - Phone Number - INT
   - Role (p1, p2, p3) - ENUM(P1,P2,P3)
   - County Name/ID - VARCHAR(255)/INT
   - Verified/Registered - BOOLEAN

2. County: County Table has ID, Name, and lat long for mapping
   - County ID (Primary Key) - INT
   - County Name - VARCHAR(255)
   - Latitude, Longitude – DECIMAL

3. Department: Department had ID and name and is linked to County ID, one county can have multiple departments
    - Department ID (Primary Key) – INT
    - Department Name (Sheriff/Security, Weather, Health, Fire) - VARCHAR/ENUM
    - County ID (Foreign Key) – INT

4. Metrics: Each metric has a type/value which indicates COVID-19 cases per 100k, covid deaths per 100k, wildfire levels, etc.

   A department can report/store multiple metrics.
    - Metric ID (Primary Key) – INT
    - Department ID (Foreign Key) – INT
    - County ID (Foreign Key) – INT
    - Metric Type – VARCHAR
    - Metric Value – DECIMAL
    - Timestamp – DATETIME

5. Alert: When a metric reaches an alert level, there is an entry in alert able (with a message), it's triggered by a director or Admin level user
    - Alert ID (Primary Key) – INT
    - Metric ID (Foreign Key) – INT
    - Alert Message – TEXT
    - Triggered By (User ID, Foreign Key) – INT
    - Timestamp - DATETIME

6. User_Alerts: User to Alert mapping. Users can receive multiple alerts, and alerts can be sent to multiple users
    - User ID (Foreign Key) – INT
    - Alert ID (Foreign Key) – INT

Storage for Video and Audio files (should it be necessary) will happen on the server and the physical path will be saved in the database to be able to locate it again. To save space the data will be converted to a Base64 string and sent in a request body.

Searching and Filtering

Below is a list of filters on per table basis:

User Table:

- List users by role
- List all users from a given county
- Show all registered users (email verified)

Metrics Table:

- Search by Metric Type
- Search by Time Period: Show metrics from the last week, month, year, etc.
- Display metrics specific to a department

Other general filters:

- Show counties with more than x COVID-19 cases per 100k
- Show recent alerts (from last x days) - departments that have most metrics in the last x days

The filters/searches will be realized with well formulated SQL commands.

APIs:

User APIs:

1. Add User
    - POST request
    - /api/user/add
    - Parameters: email, phone_number, role, county_id, encrypted password (in body)
    - Adds a new user to the User table

2. Get Users by Role
    - GET request
    - /api/user/role/{role}
    - Get users by role (p1, p2, p3)

3. Get Users by County
    - GET
    - /api/user/county/{county_id}
    - All users from a county. Metrics API

4. Add Metric
    - POST
    - /api/metrics/add
    - Parameters: metric_id, department, county, metric_type, metric_value, timestamp
    - Description: Adds a new metric the Metric table.

5. Get Metrics by Type
    - GET
    - /api/metrics/type/{metric_type}
    - metrics based on their type

6. Get Metrics by Time Period
    - GET
    - /api/metrics/timeframe?start={start_date}&end={end_date}
    - metrics within a specified time frame

7. Get Metrics by Department
    - GET
    - /api/metrics/department/{department_id}
    - metrics specific to a department

Alert APIs:

8. Get Recent Alerts
    - GET
    - /api/alerts/recent?days={x}
    - alerts from the last x days

9. Add Alert
    - POST
    - /api/alerts/add
    - Parameters: metric_id, alert_message, triggered_by
    - Adds a new alert to the Alert table

User Alerts APIs:

10. Get User Alerts
    - GET
    - /api/user-alerts/user/{user_id}
    - all alerts for a specific user

General APIs:

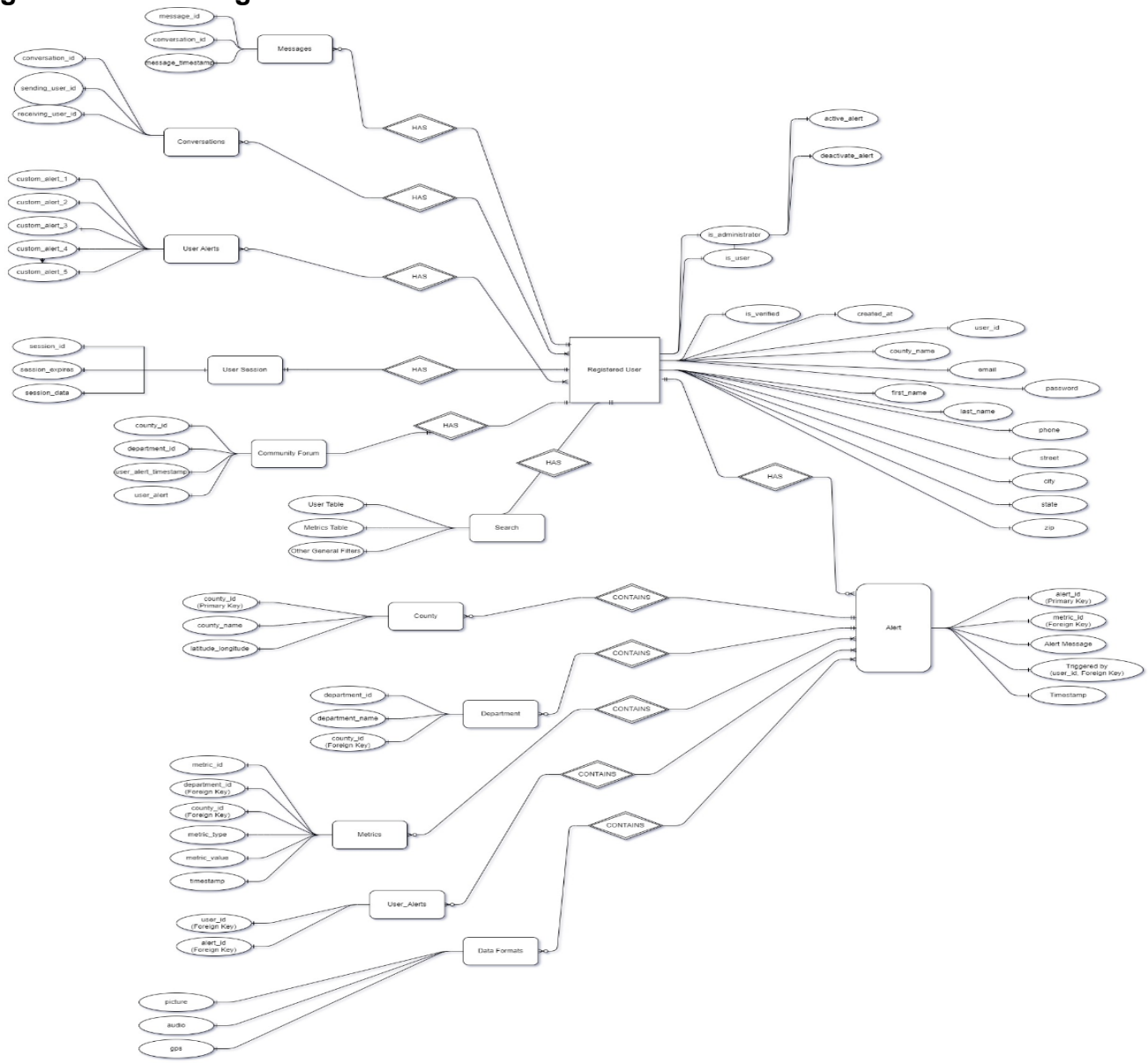
11. Get Counties by COVID-19 Cases (repeat for other types)
    - GET
    - /api/counties/covid-cases?threshold={x}
    - counties with more than x COVID-19 cases per 100k.

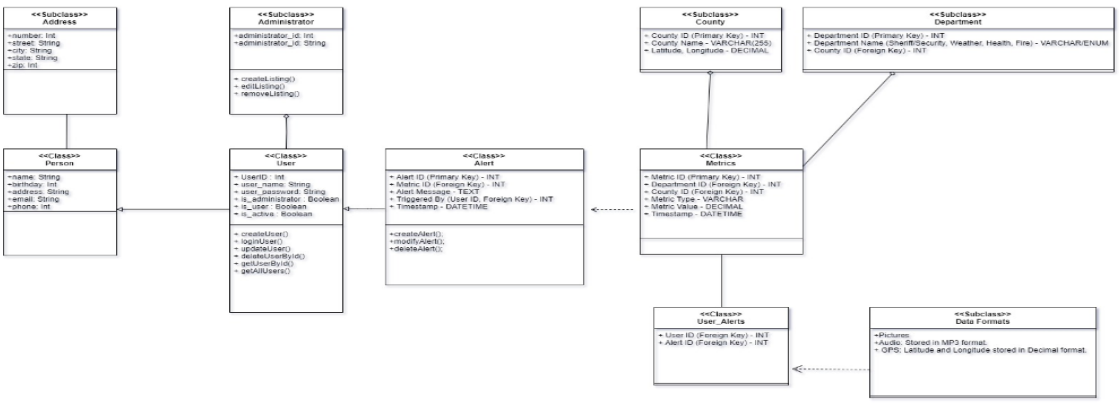
12. Get Departments with most metrics
    - GET
    - /api/departments/top-metrics?days={x}
    - departments that have reported the most metrics in the last x days

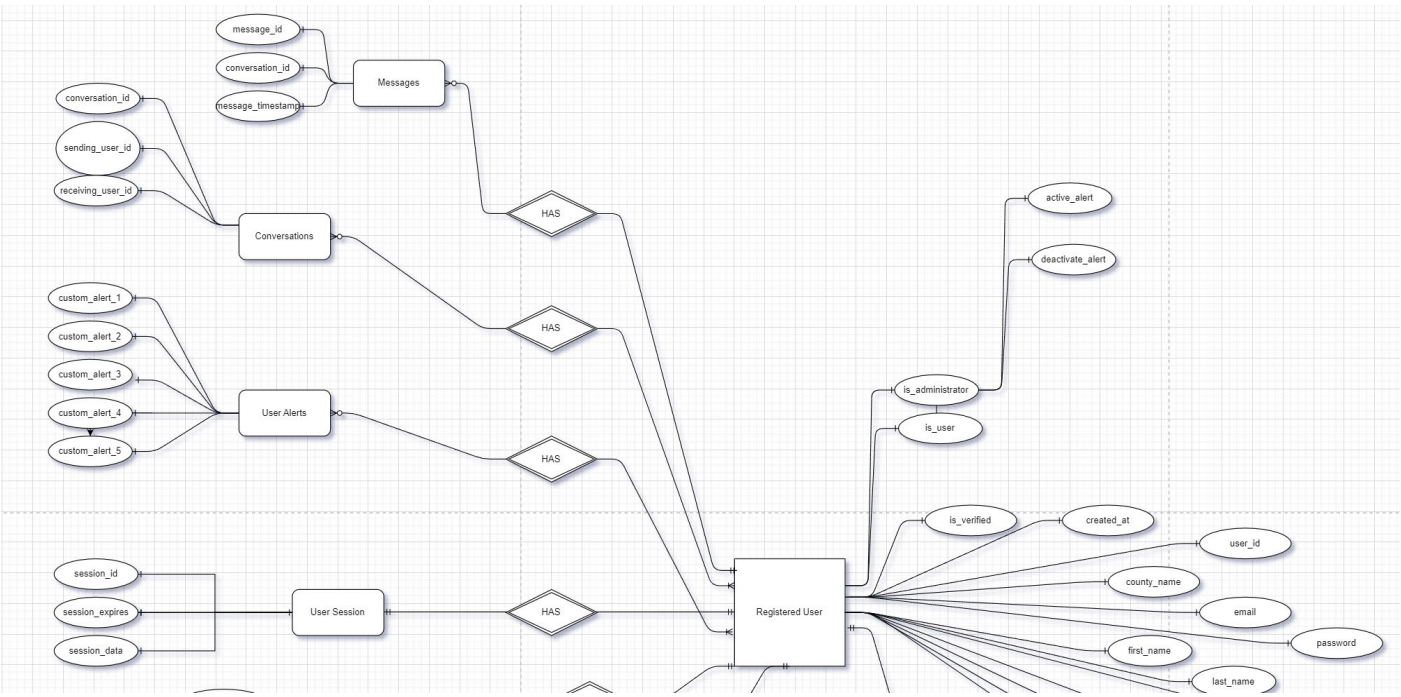
Algorithms:

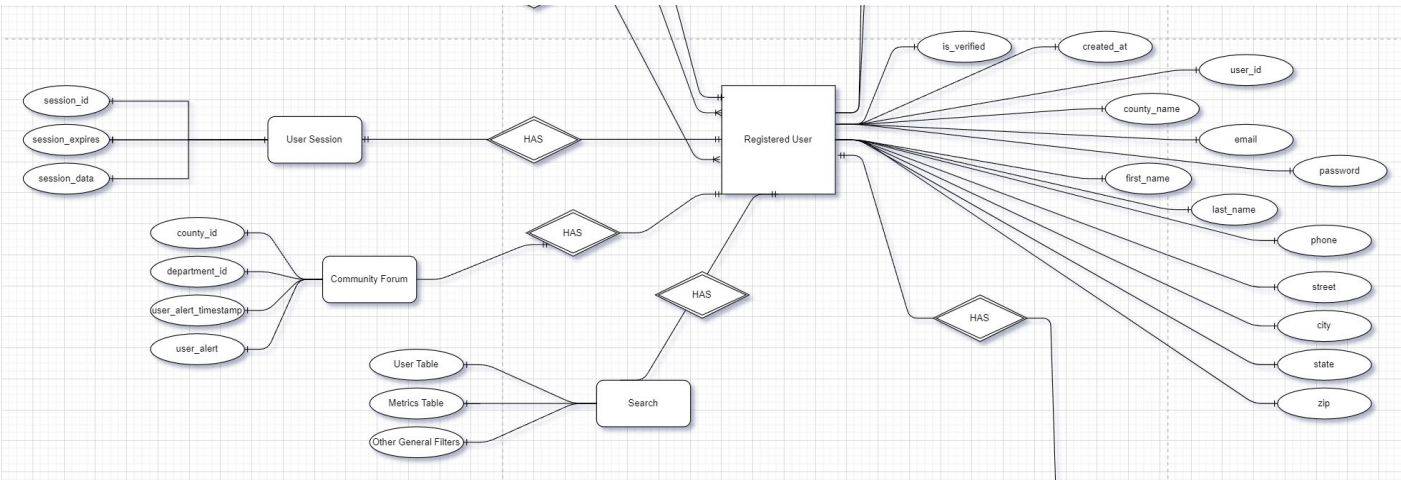
Then only special algorithm potentially being used is the Dijkstra algorithm should a feature for route calculation be included

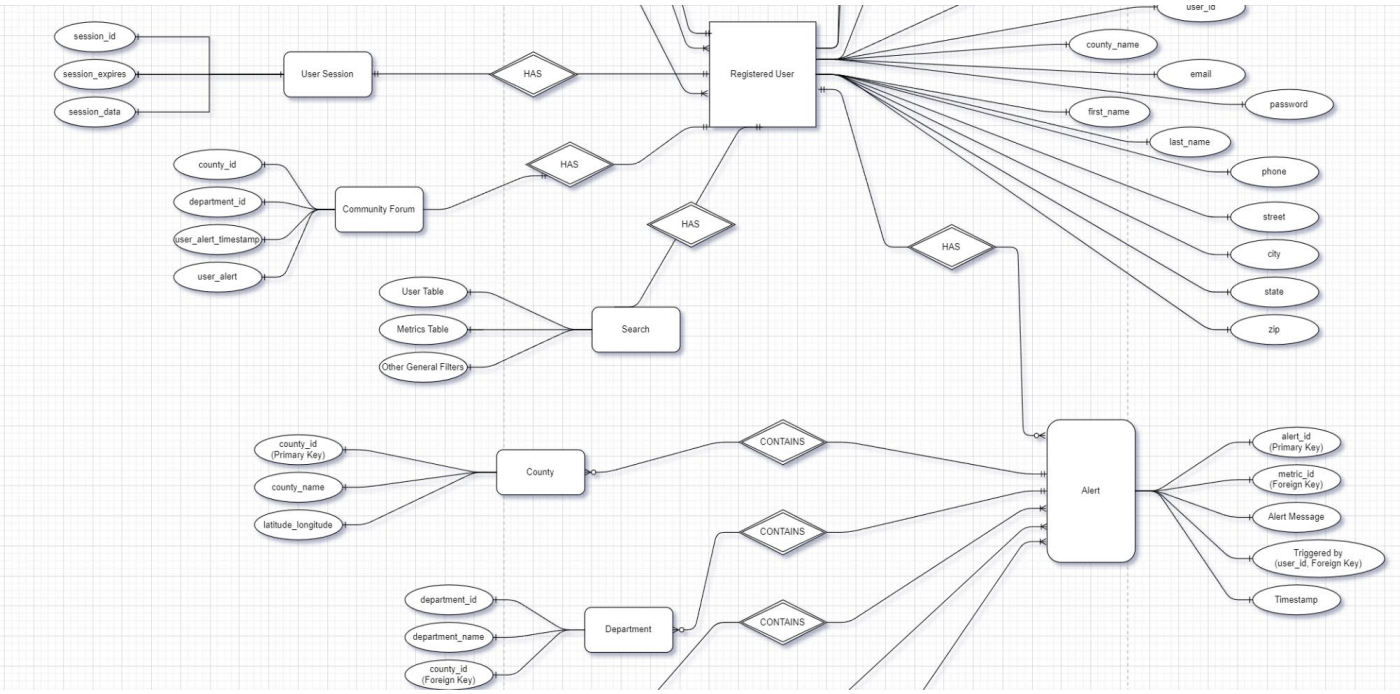
4. High Level UML Diagrams

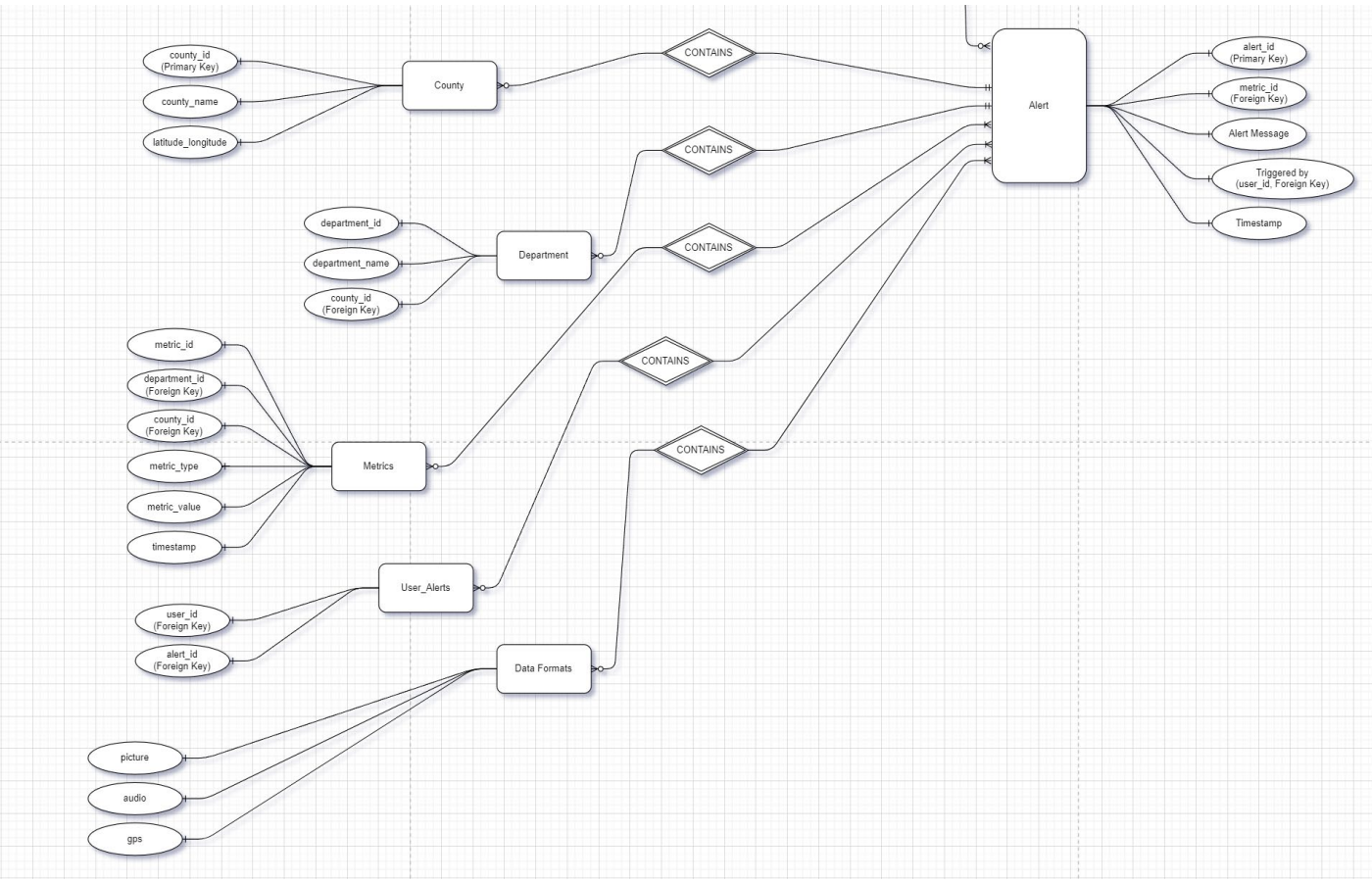


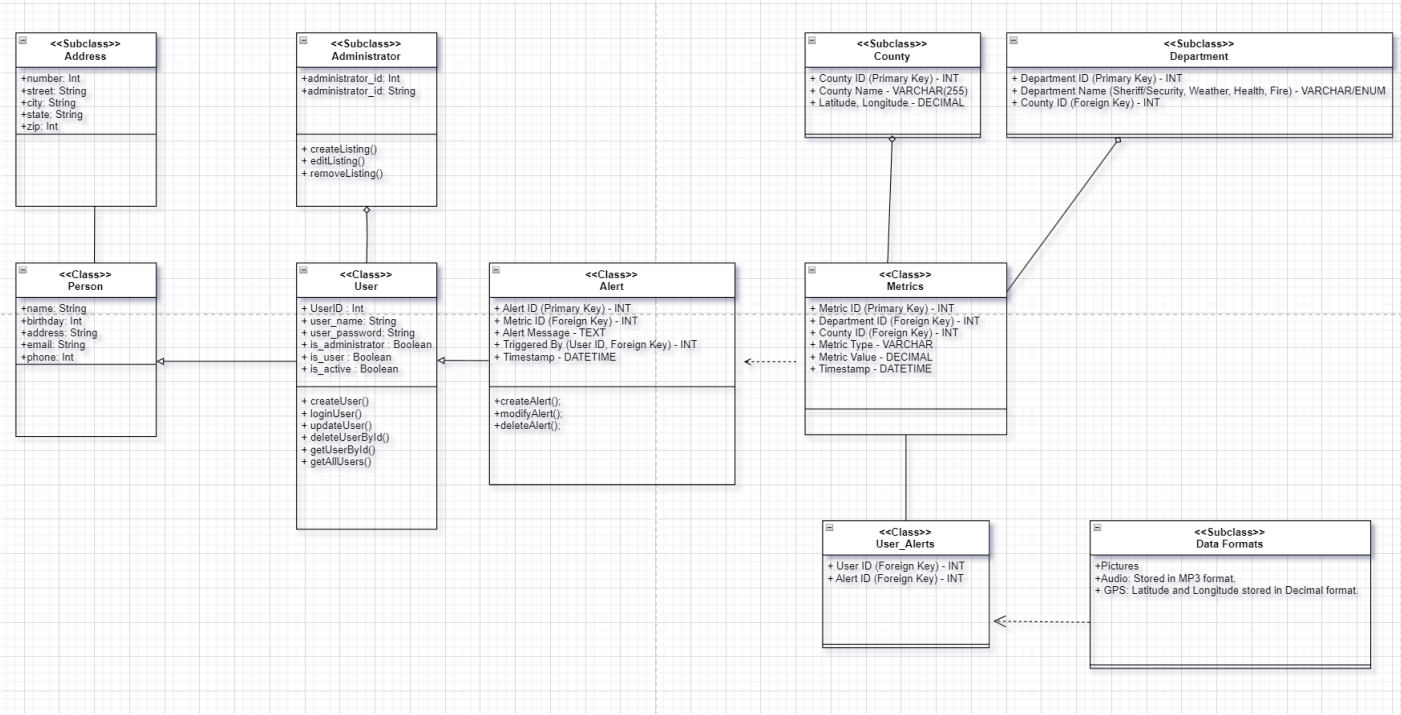



message_id
conversation_id
message_timestamp
Messages
conversation_id
sending_user_id
receiving_user_id
Conversations
HAS
HAS
HAS
active_alert
deactivate_alert
custom_alert_1
custom_alert_2
custom_alert_3
custom_alert_4
custom_alert_5
User Alerts
is_administrator
is_user
is_verified
created_at
user_id
county_name
email
password
first_name
last_name
session_id
session_expires
session_data
User Session
HAS
Registered User


session_id
session_expires
session_data
User Session
HAS
Registered User
is_verified
created_at
user_id
county_name
email
password
first_name
last_name
phone
street
city
state
zip
county_id
department_id
user_alert_timestamp
user_alert
Community Forum
HAS
HAS
HAS
User Table
Metrics Table
Other General Filters
Search


session_id
session_expires
session_data
User Session
HAS
Registered User
user_id
county_name
email
password
first_name
last_name
phone
street
city
state
zip
county_id
department_id
user_alert_timestamp
user_alert
Community Forum
HAS
HAS
HAS
User Table
Metrics Table
Other General Filters
Search
county_id
(Primary Key)
county_name
latitude_longitude
County
CONTAINS
CONTAINS
CONTAINS
department_id
department_name
county_id
(Foreign Key)
Department
Alert
alert_id
(Primary Key)
metric_id
(Foreign Key)
Alert Message
Triggered by
(user_id, Foreign Key)
Timestamp


county_id
(Primary Key)
county_name
latitude_longitude
County
CONTAINS
CONTAINS
CONTAINS
CONTAINS
CONTAINS
Alert
alert_id
(Primary Key)
metric_id
(Foreign Key)
Alert Message
Triggered by
(user_id, Foreign Key)
Timestamp
department_id
department_name
county_id
(Foreign Key)
Department
metric_id
department_id
(Foreign Key)
county_id
(Foreign Key)
metric_type
metric_value
timestamp
Metrics
User_Alerts
user_id
(Foreign Key)
alert_id
(Foreign Key)
Data Formats
picture
audio
gps



Architecture and Organization

1. Registered User
    a. Attributes
        i. is_verified
        ii. created_at
        iii. user_id
        iv. county_name
        v. email
        vi. password
        vii. first_name
        viii. last_name
        ix. phone
        x. street
        xi. city
        xii. state
        xiii. zip
        xiv. is_user
        xv. Is_administrator
    b. Relationship
        i. search
        ii. community forum
        iii. user sessions
        iv. user alerts
        v. conversations
        vi. messages
        vii. alert

2. Search
    a. Attributes
        i. user table
        ii. metrics table
        iii. other general filters
    b. Relationship
        i. user
        ii. alert
        iii. metrics
        iv. county
        v. department
        vi. user_alerts
        vii. data formats
3. Community Forum
    a. Attributes
        i. County_id
        ii. Department_id
        iii. User_alert_timestamp
        iv. User_alert
    b. Relationship
        i. user
        ii. alert
        iii. metrics
        iv. county
        v. department
        vi. user_alerts
        vii. data formats
4. User Session
    a. Attributes
        i. session_id
        ii. session_expries
        iii. session_data
    b. Relationship
        i. user
5. User Alerts
    a. Attributes
        i. custom_alert_1
        ii. custom_alert_2
        iii. custom_alert_3
        iv. custom_alert_4
        v. custom_alert_5
    b. Relationship
        i. user
        ii. alert
6. Conversations

    a. Attributes
        i. converstion_id
        ii. sending_user_id
        iii. receiving_user_id
    b. Relationships
        i. user
        ii. messages
7. Messages
    a. Attributes
        i. message_id
        ii. conversation_id
        iii. message_timestamp
    b. Relationship
        i. converstions
8. Alert
    a. Attributes
        i. alert_id (primary key)
        ii. metric_id (foreign key)
        iii. alert message
        iv. triggered by (user_id, foreign key)
        v. Timestamp
    b. Relationships
        i. County
        ii. Department
        iii. Metrics
        iv. User_Alerts
        v. Data Formats
9. County
    a. Attributes
        i. county_id (primary key)
        ii. county_name
        iii. latitude_longitude
    b. Relationships
        i. alert
        ii. user
10. Department
    a. Attributes
        i. department_id
        ii. department_name
        iii. county_id (foreign key)
    b. Relationships
        i. alert
        ii. user
11. Metrics
    a. Attributes

        i. metric_id (primary key)
        ii. department_id (foreign key)
        iii. metric_type
        iv. metric_value
        v. timestamp
    b. Relationships
        i. alert
        ii. user
12. User_alerts
    a. Attributes
        i. user_id (foreign key)
        ii. alert_id (foreign key
    b. Relationships
        i. alert
        ii. user
13. Data Formats
    a. Attributes
        i. picture
        ii. audio
        iii. gps
    b. Relationships
        i. alert
        ii. user

5. Risks

There are two key risks we identified:

1. schedule
2. skill

Many team members are very busy and have long commutes which prevents them from working as much as necessary on the assignments. This in turn leads to a risk of skill since they also don't have the time to learn the required skills (should they not already have it), a possible solution is to move the out of class meeting online and forcing everyone to actually come to class. The use of Trello is also a key solution I identified to help everyone keep track of things.

6. Project management

Starting with this milestone we’re starting to work with a modified scrum methodology that works in sprints with the duration of the next milestone. After each milestone we’ll discuss things in a retrospective and then act accordingly. To visualize the sprint progress we’ll use Trello, this will also help with keeping track of everyone’s work and ensures that things that are not on track are noticed as soon as possible. Every task will be designated as frontend, backend, or database, from there any team member of this sub team can take the task and start working on it. This works a little differently than for Milestone 2, where very specific work was assigned to each team member and then closely monitored by the team lead following the experience Milestone 1.

Additionally, frontend and backend teams are encouraged to work independently and discuss things with each other without having to involve the team lead if not for important points of discussion.

This should help to ensure that everything is being completed in time and ready to being presented when necessary.